

예외 → HTTP 번역, 심화 복기 (6문답)

(어제의 9줄을 "확인"이 아니라 "한 층 아래"로 밀어본 날 — 1맞음 5모름, 그 5가 좌표다)

한눈에

항목	내용
성격	심화 복기 (새 코드 0줄, 개념 6자리) — 시간 제약 세션
왜 이 방식	어제 파일의 검증질문 10개는 <i>확인/증(배운 걸 다시 꺼내기)</i> . 그것만으론 커버가 안 된다고 판단 → 한 층 아래를 파는 질문 6개 를 새로 세워 풀었다
결과	Q3 정답(500) · 나머지 5문항 즉답 실패 → 모범답 정리
★ 오늘의 획득	4xx / 5xx의 진짜 경계선 = "규칙이 정상 작동한 거절"인가, " 우리 코드가 틀렸다는 증거 "인가
드러난 구멍 5	@RestControllerAdvice (핸들러 중복) · 에러 본문 JSON 구조화 · 역동성 응답 설계 · 예외 메시지 노출과 보안 · "없을 때"의 공통 패턴
다음	핸들러 2개(없는 계좌·amount≤0) — 어제 패턴 복제, 10분거리 . 그 다음 m106

어제 500을 400으로 바꾸고 "예외 번역을 배웠다"고 적었다. 오늘 그 아래를 파보니 — 배운 건 **패턴 하나**였고, 그 패턴이 서 있는 **판단의 지반**은 아직 절반이 비어 있었다. 6문항 중 1개만 즉답됐다는 사실이 그 증거다. 그런데 맞힌 그 하나(Q3)가 하필 지반의 중심이었다.

0. 오늘의 방식 — 왜 "확인"이 아니라 "심화"인가

어제 작업기록의 검증질문 10개는 **확인형**이다: "@ExceptionHandler의 방아쇠는 무엇인가", "400과 409 중 400을 고른 근거는" — 어제 이미 답이 나온 것들을 다시 꺼내는 훈련. 그것도 필요하지만, 오늘은 다른 걸 했다. **어제 답이 나오지 않은 자리**, 즉 어제 코드를 짜면서 *지나쳤지만 언젠가 반드시 마주칠* 질문 6개를 세우고 풀었다.

	어제의 검증질문 10개	오늘의 심화 6문항
성격	확인 — 배운 것을 꺼내기	확장 — 배운 것의 <i>한 층 아래/옆</i>
예	"방아쇠가 예외인 애노테이션은?"	"컨트롤러가 3개면 그 9줄을 3벌 복사하나?"
실패의 의미	잊었다	아직 안 배웠다 = 다음 좌표

결과: 1맞음 / 5모름. **5개의 "모름"이 오늘의 실질 산출물**이다 — 모르는 자리를 정확히 아는 것이 다음 학습의 좌표가 된다.

1. 기준 코드 (어제 완성분 — 오늘 질문의 대상. 풀코드 병기 규칙)

```
package com.ledger.api;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;

@RestController
public class TransferController {

    private final Bank bank = new Bank();

    public TransferController() {
        bank.createAccount("a", "이름", 2100);
        bank.createAccount("b", "이름", 3500);
        bank.createAccount("platform", "플랫폼", 0);
    }

    @PostMapping("/transfer")
    public String transfer(@RequestBody TransferRequest request) {
        bank.transfer(request.fromId(), request.toId(), request.amount(), request.idempotencyKey());
        return "a: " + bank.getAccount("a").getBalance()
            + ", b: " + bank.getAccount("b").getBalance()
            + ", platform: " + bank.getAccount("platform").getBalance();
    }

    // 어제 추가한 9줄 — 오늘 심화의 대상
    @ExceptionHandler(InsufficientBalanceException.class)
    public ResponseEntity<String> handleInsufficientBalance(InsufficientBalanceException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("이체 실패: " + e.getMessage());
    }
}
```

PART 1 — ★ Q3: 4xx / 5xx의 진짜 경계선 (오늘 유일하게 맞힌 것, 그리고 가장 중요한 것)

질문

checkBalance() 가 "돈 보존 실패"를 던진다면 — 즉 이체 후 전체 잔액 합이 0이 아니라면 — 이걸 **400인가 500인가?** 잔액 부족과 근본적으로 무엇이 다른가?

내 답

500. (정답)

왜 500인가 — 두 실패의 성격이 완전히 다르다

	잔액 부족 (여제 400)	돈 보존 실패 (500)
무슨 일이 일어났나	a가 2100원인데 99999를 요청	이체가 끝났는데 전체 합이 안 맞음
시스템은?	정상 작동했다 — 규칙대로 정확히 거절	고장났다 — 있을 수 없는 일이 일어남
누구 탓	클라이언트 (무리한 요청)	나 (내 코드에 버그)
고치는 사람	클라이언트가 요청을 고침	내가 코드를 고침
클라이언트가 재시도하면	잔액은 그대로 → 또 실패 (무의미)	재시도하면 안 됨 — 상태가 이미 오염됐을 수 있음
상태코드	4xx (400)	5xx (500)

핵심 한 문장:

잔액 부족은 **규칙이 정상 작동한 결과**이고, 돈 보존 실패는 **규칙이 깨졌다는 증거**다.

전자는 "당신의 요청이 우리 규칙에 걸렸습니다"(= 예상된 실패, 설계의 일부). 후자는 "우리 시스템이 자기 규칙조차 못 지켰습니다"(= 예상 못 한 실패, 버그).

불변식(invariant)이라는 개념 — 오늘 언어로 잡은 것

복식부기의 "합 = 0"은 불변식이다: 어떤 경우에도 참이어야 하는 **명제**. 시스템이 정상인 한 절대 깨지지 않아야 한다.

- 잔액 부족 예외 = **입력 검증 실패**. 나쁜 입력을 막는 문 → 이 문은 *열심히 작동할수록* 정상이다.
- 돈 보존 실패 = **불변식 위반**. 있을 수 없는 일이 일어난 것 → 이게 터졌다는 건 코드가 틀렸다는 뜻이다.

그래서 이 둘은 "잡는 방식"조차 달라야 한다:

- 잔액 부족(400) → 클라이언트에게 사유를 **친절히 알려준다**("잔액이 충분하지 않습니다"). 클라이언트가 행동을 고칠 수 있으니까.
- 돈 보존 실패(500) → 클라이언트에게는 **자세히 말하지 않는다**(어차피 클라이언트가 할 수 있는 게 없다). 대신 **서버 로그에 크게 남기고, 개발자에게 알림이 가야 한다**. 실무라면 이걸 **페이지(호출) 감**이다 — 새벽 3시에 담당자를 깨우는 종류의 오류다. 결제 시스템에서 돈이 안 맞는다는 건 최고 등급 사고다.

이 경계선을 판정하는 질문 (앞으로 쓸 도구)

새 예외를 만날 때마다 스스로 묻는다:

"이게 터졌을 때, 고쳐야 할 사람은 누구인가?"
- 클라이언트가 요청을 고치면 되는가 → 4xx
- 내가 코드를 고쳐야 하는가 → 5xx

잔액 부족 → 클라이언트가 금액을 줄이면 된다 → 400.

돈 보존 실패 → 클라이언트가 뭘 해도 소용없다, 내가 코드를 고쳐야 한다 → 500.

없는 계좌 → 클라이언트가 id를 고치면 된다 → 4xx. (그래서 다음 세션의 핸들러가 4xx인 것)

연결: 7/5에 원자성 버그를 잡을 때, checkBalance를 transfer 끝에서 실제로 호출하게 만든 이유가 바로 이것이다 — **불변식은 선언만 하면 장식이고, 매번 검사해야 안전망이 된다**. 그 안전망이 울린다면 그건 클라이언트 잘못이 아니라 **내 코드가 틀렸다는 알람**이다. 오늘 그 알람이 HTTP에서 무슨 색이어야 하는지(500)까지 정해졌다.

PART 2 — 즉답 실패 5자리 (모범답 + 다음 좌표)

아래 다섯은 "모름"이었다. 정직하게 기록한다. 모른다는 것을 아는 게 다음 학습의 좌표이므로.

Q1. 컨트롤러가 3개로 늘면 그 9줄을 3별 복사해야 하나?

답: 아니다. 지금 구조의 한계가 맞고, 스프링의 해법은 @RestControllerAdvice 다.

지금 구조의 한계: @ExceptionHandler를 컨트롤러 안에 두면, 그 핸들러는 그 컨트롤러에서 터진 예외만 잡는다. 컨트롤러가 TransferController / AccountController / RefundController 셋이 되면 — 같은 InsufficientBalanceException을 잡는 9줄을 세 별 복사해야 한다. 그리고 어느 날 400을 422로 바꾸기로 하면 세 군데를 다 고쳐야 한다. 한 군데를 빠뜨리면 같은 오류가 API마다 다른 코드로 나가는 최악의 상태가 된다.

이건 새로운 종류의 문제가 아니다 — 7/4에 만난 매직 넘버와 같은 병이다. 그때 feeRate를 필드로 뺐 이유가 "한 곳에서 바꾸면 다 반영되게"였다. 예외 처리 정책도 똑같이 한 곳에 있어야 한다.

해법: 예외 핸들러들을 컨트롤러 밖의 전담 클래스로 빼고, 거기에 @RestControllerAdvice를 붙인다. 스프링은 부팅 시 이 명칭을 보고 "이 클래스의 핸들러들은 모든 컨트롤러에 공통 적용"으로 등록한다.

```
package com.ledger.api;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice          // ← "전역 예외 처리 담당"이라는 명칭
public class GlobalExceptionHandler {

    @ExceptionHandler(InsufficientBalanceException.class)
    public ResponseEntity<String> handleInsufficientBalance(InsufficientBalanceException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("이체 실패: " + e.getMessage());
    }

    // 컨트롤러가 몇 개든, 이 한 벌이 전부 커버한다
}
```

7/10에 판 명찰 원리로 읽으면: `@RestController` 명찰 = "나는 HTTP 창구". `@RestControllerAdvice` 명찰 = "나는 전 창구 공통 규정집". 부팅 때 스프링이 이 명찰을 보고 예외 표를 만들 때, 특정 컨트롤러가 아니라 전역에 등록한다는 점만 다르다.

그럼 지금 당장 옮겨야 하나? → 아니다. 지금 컨트롤러는 하나뿐이라 중복이 0이다. **중복이 실제로 생길 때 옮기는 게 맞다**(성급한 추상화 금지). 다만 **알고 안 하는 것과 몰라서 안 하는 것은 다르다** — 오늘 그 차이를 없앴다. 컨트롤러가 들어 되는 순간이 이걸 꺼내는 순간이다.

Q2. 에러 본문을 문자열이 아니라 JSON 구조로 주는 이유는?

답: 받는 쪽(클라이언트 코드)이 그것을 *읽어서 보기*해야 하기 때문이다. 문자열은 기계가 못 읽는다.

지금 우리 400의 본문:

```
이체 실패: 잔액이 충분하지 않습니다
```

여제 봤던 스프링 기본 500의 본문:

```
{"timestamp": "2026-07-12T11:08:19.639Z", "status": 500, "error": "Internal Server Error", "path": "/transfer"}
```

앱 개발자 입장에서 생각해보자. 이 API를 쓰는 앱은 400을 받으면 사용자에게 뭔가 보여줘야 한다. 그런데:

- **문자열이면** — 앱 코드가 `if (body.contains("잔액"))` 같은 **문자열 검사**를 해야 한다. 이걸 끄떡하다: ① 내가 메시지 문구를 "잔액이 부족합니다"로 살짝 바꾸는 순간 앱이 조용히 깨진다 ② 영어 앱은 어떻게 하나(다국어) ③ 오타 하나에 분기가 무너진다. **응답 문구가 실질적인 API 계약이 되어버린다.**
- **JSON이면** — 앱은 `error.code == "INSUFFICIENT_BALANCE"` 로 **안정적으로 분기**한다. 문구(message)는 바뀌어도 코드(code)는 안 바뀌니까.

실무 에러 응답의 전형:

```
{
  "code": "INSUFFICIENT_BALANCE",
  "message": "잔액이 충분하지 않습니다",
  "timestamp": "2026-07-13T11:20:00Z"
}
```

- `code` — 기계가 읽는다(분기용, 불변, 대문자 상수 스타일)
- `message` — 사람이 읽는다(로그·디버깅용, 바뀔 수 있음)

연결 — 이젠 6/27의 그 결정과 같은 종류다. 그때 거래 종류를 String "deposit" 이 아니라 **enum**으로 만든 이유: 문자열은 오타가 조용히 통과하지만 enum은 컴파일 단계에서 막힌다. 여기서도 같다 — 기계가 판단할 것은 자유 문자열이 아니라 **정해진 코드여야 한다**. 계층만 바뀌었지 원칙은 동일하다.

지금 당장 바꾸나? → 다음 좌표로 돈다. 핸들러 2개(없는 계좌·amount≤0)를 추가하면 에러 종류가 3개가 된다 — 그때가 **에러 응답을 구조화(record ErrorResponse + code)할 자연스러운 시점**이다. TransferRequest를 record로 만들었듯, ErrorResponse도 record 한 줄이면 된다.

Q3. → PART 1 참조 (유일한 정답)

Q4. 멍등성 스킵을 200 + 잔액으로 돌려주는 게 맞나?

답: "돈이 두 번 안 빠진다"는 관점에선 맞다. 그러나 클라이언트가 "새로 처리됨"과 "이미 처리돼서 스킵됨"을 구분할 수 없다는 문제가 남는다. 그리고 Stripe는 구분할 수 있게 해준다.

지금 동작: 같은 키로 재요청 → `transfer()` 첫 줄에서 `return;` → 컨트롤러는 (변하지 않은) 잔액 문자열을 200으로 반환. 클라이언트가 보는 응답은 첫 요청과 구별이 안 된다.

이게 왜 문제인가 — 그리고 왜 문제가 아닌가:

- **문제가 아닌 쪽(설계 의도):** 멍등성의 목적이 정확히 이것이다. 타임아웃 후 재시도한 클라이언트는 "성공했구나"라는 **같은 결과**를 받아야 한다. 매번 다른 답이 오면 재시도가 무서워진다. "**몇 번을 보내도 결과는 하나**"가 계약이므로, 응답도 하나여야 자연스럽다.
- **문재인 쪽(관측 가능성):** 그런데 서버 운영자·클라이언트 개발자 입장에서 구분이 필요할 때가 있다. "지금 중복 요청이 얼마나 들어오고 있나?"(네트워크가 불안정하다는 신호일 수 있다), "이 응답이 방금 처리된 건가 캐시된 건가?"(디버깅). 지금 구조에선 이걸 알 방법이 없다.

Stripe는 어떻게 하나 (추측이 아니라 실제 관행): Stripe는 멍등 재요청에도 **원래의 응답을 그대로 다시 돌려준다**(응답까지 저장해둔다). 그리고 그 응답이 **재생된 것임을 헤더로 알려준다**(`Idempotent-Replayed: true` 성격의 신호). 즉 결과는 같게, 사실은 구분 가능하게. 둘 다 취하는 설계다.

우리 구조의 더 큰 구멍 — 이게 진짜다: Stripe는 "원래의 응답"을 돌려주지만, 우리는 "**지금의 잔액**"을 돌려준다. 이 둘은 다르다:

1. key-1로 이제 → 응답 "a: 1080"
2. 다른 요청들이 오가며 a의 잔액이 변함 (예: a: 60)
3. key-1 재전송 (타임아웃 재시도) → 우리 응답: "a: 60" ← 첫 응답(1080)과 다르다!

돈은 두 번 안 빠졌으니 **역등성의 핵심(부작용 1회)**은 **지켜졌다**. 그러나 **응답의 역등성은 안 지켜졌다** — 같은 요청에 다른 답이 나갔다. 엄밀한 역등성 API는 **응답까지 저장해서 그대로 재생**한다.

→ **다음 좌표**: 지금 `Set<String> usedKeys` 는 "본 키"만 기억한다. 응답까지 역등하게 하려면 `Map<String, 응답>` 으로 바꿔 **"키 → 그때의 응답"을 저장**해야 한다. (6/29에 "지금 transfer가 void라 스킵이 맞다"고 판단했던 그 트레이드오프가, 컨트롤러가 응답을 만드는 지금 다시 돌아온 것이다. 그때의 판단은 그때 맞았고, 계층이 하나 늘어난 지금 재검토 대상이 됐다.)

Q5. `e.getMessage()` 를 그대로 클라이언트에 노출하는 게 항상 안전한가?

답: 아니다. 지금은 안전하지만(내가 쓴 도메인 메시지라서), 이 습관 자체가 위험하다. 예외 메시지에는 내부 정보가 담기기 쉽다.

지금이 안전한 이유: `InsufficientBalanceException` 의 메시지("잔액이 충분하지 않습니다")는 내가 6/25에 직접 `super(message)` 로 심은 문장이다. 클라이언트에게 보여주려고 쓴 말이니 노출돼도 문제없다.

위험해지는 경우 — 예외가 내가 안 쓴 것일 때:

시스템 예외(DB·파일·네트워크)의 메시지는 **내부 구조를 그대로 뱉는다**. 예를 들어(8월 이후 실제로 만날 것들):

- DB 예외: `Table 'ledger_prod.accounts' doesn't exist` → **DB 이름·테이블 구조가 노출**된다
- 연결 예외: `Connection refused: 10.0.1.42:5432` → **내부 서버 IP와 포트가 노출**된다
- 파일 예외: `/opt/ledger/secrets/wal.dat (Permission denied)` → **서버 파일 경로가 노출**된다

이 정보들은 공격자에게 **정찰(reconnaissance)** 자료다. 시스템 구조를 알려주는 것이니까. 실제로 "에러 메시지에 스택트레이스나 내부 경로가 그대로 나오는 서버"는 보안 점검에서 지적당하는 고전적 항목이다.

원칙:

4xx(클라이언트 오류)의 메시지는 친절하게, 5xx(서버 오류)의 메시지는 감춘다.

- 4xx → 내가 쓴 도메인 메시지 노출 OK. 클라이언트가 행동을 고치려면 이유를 알아야 한다.

- 5xx → 클라이언트에게만 일반 메시지("처리 중 오류가 발생했습니다"). **진짜 내용(스택트레이스·원인)**은 **서버 로그**에 남긴다. 클라이언트는 어차피 아무것도 할 수 없고, 자세히 알려줘봐야 정보만 샌다.

Q3와 정확히 맞물린다: 돈 보존 실패(500)를 처리할 때 `e.getMessage()` 를 그대로 내보내면 안 된다는 뜻이다 — 그건 내부 불변식이 깨졌다는 정보다. **밖에는 "처리 중 오류", 안(로그·알림)에는 전부. 4xx와 5xx는 상태코드만 다른 게 아니라 정보 공개 정책 자체가 다르다.**

Q6. "표가 비어 있으면 500"과 "Map.get이 null을 돌려준다" — 공통 패턴은?

답: 둘 다 "찾았는데 없다"를 다루는 방식이고, 공통 패턴은 이것이다 — 찾기에 실패하면 시스템은 폭발하지 않고 **미리 정해둔 기본 동작**으로 떨어진다. 그 기본값을 알고 쓰지 않으면 조용히 물린다.

	6/22 <code>Map.get</code>	7/12 예외 핸들러 표
찾는 것	키에 해당하는 값	예외에 해당하는 핸들러
없으면	null 반환 (예외 아님)	기본 동작 = 500 (프로그래밍 종료 아님)
위험	"없으면 알아서 터지겠지"라 믿으면 → 조용히 null이 흘러가 엉뚱한 데서 NPE	"예외 던졌으니 알아서 되겠지"라 믿으면 → 조용히 500이 나가 API가 거짓말
해법	내가 null을 검사해서 의미 있는 예외로 바꾼다 (<code>AccountNotFoundException</code>)	내가 핸들러를 등록해서 의미 있는 상태코드로 바꾼다 (400)

한 문장:

"찾는 게 없을 때"의 기본값은 언제나 존재하고, 그 기본값은 대개 내가 원하는 답이 아니다. 기본값에 기대지 말고 명시적으로 처리하라.

6/22에 `accounts.get(id)` 가 null을 주는 걸 몰랐다면 — 없는 계좌로 이체할 때 "알아서 터지겠지" 하고 넘어갔다가, 한참 뒤 엉뚱한 곳에서 NPE로 만났을 것이다. 그래서 **null을 직접 검사해서 `AccountNotFoundException` 으로 바꿨다.**

7/12에 핸들러 표가 비면 500이 나가는 걸 몰랐다면 — "예외를 던졌으니 뭔가 적절히 처리되겠지" 하고 넘어갔다가, 클라이언트가 무의미한 재시도를 도는 걸 몰랐을 것이다. 그래서 **핸들러를 등록해서 400으로 바꿨다.**

같은 교훈의 두 계층이다. 6월엔 자바 안에서, 7월엔 HTTP 경계에서. 그리고 8월에도 또 만날 것이다 — 파일에 데이터가 없을 때, 로그에 레코드가 없을 때. **"없을 때 뭐가 되나"**는 항상 물어야 하는 질문이다.

PART 3 — 오늘 얻은 판정 도구

앞으로 새 예외를 만날 때마다 쓸 **3단 질문**:

- ① 이게 터졌을 때, 고쳐야 할 사람은 누구인가?
클라이언트가 요청을 고치면 됨 → 4xx
내가 코드를 고쳐야 함 → 5xx (← Q3의 경계선)
- ② 클라이언트에게 이유를 알려줘도 되는가?
4xx → 도메인 메시지 노출 OK (행동을 고칠 수 있어야 하니까)
5xx → 감춘다. 밖에 일반 메시지, 안(로그)엔 전부 (← Q5)
- ③ 클라이언트 코드가 이걸 읽고 분기해야 하는가?
그렇다 → 자유 문자열이 아니라 `code`를 담은 JSON으로 (← Q2)

4. 산출물 / 다음

- **산출:** 이 작업기록(심화 6문답). 새 코드는 0줄 — **오늘은 지반을 판 날**이다.
- **드러난 좌표 5개** (전부 "언제 꺼낼지"까지 정해둠):

좌표	꺼낼 시점
@RestControllerAdvice 로 핸들러 분리	컨트롤러가 돌이 되는 순간 (지금은 중복 0이라 불필요)
에러 응답 JSON 구조화 (record ErrorResponse(code, message))	핸들러가 3개 가 되는 다음 세션이 자연스러운 시점
역등성 응답 재생 (Set → Map<키, 응답>)	응답의 역등성을 논할 때. 6/29의 트레이드오프가 계층이 늘어 돌아온 것
불변식 위반(500) 전용 처리 + 로그·알림	checkBalance 실패를 실제로 핸들링할 때
5xx 메시지 은닉	위와 동시

- **다음 세션 (10분거리):** 핸들러 2개 — **없는 계좌 → 4xx**(AccountNotFoundException), **amount ≤ 0 → 400**(IllegalArgumentException). 어제 9줄 패턴 복제. 오늘 판 것 덕에 "왜 4xx인지" (①: 클라이언트가 id/금액을 고치면 된다)를 근거로 말할 수 있다.
- **그 다음:** m106 CodeCrafters Redis 1단계 · 7월 말 m107 점검.

5. 검증 질문 (답 적지 말 것)

1. 잔액 부족과 돈 보존 실패의 차이를 "고쳐야 할 사람"으로 설명하라. 각각 몇 xx인가?
2. **불변식(invariant)**이란 무엇인가? 복식부기의 합=0이 왜 불변식인가? 그것이 깨졌다는 건 무슨 뜻인가?
3. 컨트롤러가 3개일 때 @ExceptionHandler 를 컨트롤러 안에 두면 생기는 문제 셋은? 해법의 이름과, 그 명찰이 스프링에게 시키는 일은?
4. 에러 본문을 문자열로 주면 앱 개발자는 무엇을 하게 되나? 그게 왜 위험한가? code 와 message 의 역할 차이는?
5. 지금 역등성 재요청 응답의 구멍은? Stripe는 무엇을 저장해서 그걸 막나? 우리가 바꿔야 할 자료구조는?
6. e.getMessage() 노출이 위험해지는 경우의 예를 둘 들라. 4xx와 5xx의 정보 공개 정책은 어떻게 다른가?
7. Map.get 의 null과 "핸들러 표가 비었을 때의 500"의 공통 패턴을 한 문장으로.

6. 회고

어제 500을 400으로 바꾸고 "예외 번역을 배웠다"고 썼다. 오늘 그 아래를 여섯 번 파봤더니 — 하나만 답할 수 있었다.

그런데 이게 나쁜 결과가 아니다. **어제 배운 건 "패턴 하나"였고, 그 패턴이 서 있어야 할 지반은 아직 안 판던 것뿐이다**. 어제의 검증질문 10개는 전부 답할 수 있다(그건 어제 답이 나온 것들이니까). 오늘 물은 건 **어제 답이 나오지 않은 자리**였고, 5개가 비었다는 건 그 자리들이 실제로 비어 있었다는 뜻이다. 모르는 걸 정확히 아는 것 — 그게 오늘의 산출물이다.

맞힌 하나(Q3)가 하필 중심이었던 게 다행이다. **"잔액 부족은 규칙이 정상 작동한 것, 돈 보존 실패는 규칙이 깨진 것"** — 이 부분이 4xx/5xx의 진짜 경계선이고, 앞으로 만날 모든 예외에 이 자를 댈 수 있 다. 500이라고 답할 때 근거를 다 말하진 못했지만, 감각은 정확히 그 자리를 짚고 있었다. 감각이 먼저 서고 언어가 뒤따라오는 것 — 오늘 그 언어(불변식, 고쳐야 할 사람)를 붙였다.

그리고 다섯 개의 "모름"이 전부 **다음 코드의 좌표**로 바뀌었다. @RestControllerAdvice는 컨트롤러가 돌이 될 때, 에러 JSON은 핸들러가 셋이 될 때, 역등 응답 재생은 응답 역등성을 논할 때 — **언제 꺼 낼지까지 정해두니 "모름"이 "예약"이 됐다**. 특히 Q4는 아프다: 6/29에 "지금 transfer가 void라 스킵이 맞다"고 내린 판단이, 컨트롤러가 응답을 만들게 된 지금 재검토 대상이 됐다. 그때 판단은 그때 맞았지만, 계층이 하나 늘면 전제가 바뀐다. 설계 결정에 유통기한이 있다는 걸 처음 실감했다.

새 코드 0줄인 날이지만, 다음에 짤 코드가 이해된 코드가 될 것이다. 7/10에도 같은 말을 썼고, 7/12이 그걸 증명했다.

한 줄: 6번 파서 1번 맞았다 — 그 하나(4xx/5xx의 경계선 = 규칙이 작동한 거절인가, 규칙이 깨진 증거인가)가 지반이었고, 나머지 다섯 개의 "모름"은 전부 좌표가 됐다.